

✓ PROJETO COMPLETO — Inventário do que foi implementado

Este documento lista **tudo o que está implementado e funcional** no backend da Ludoteca, para dar clareza total sobre o estado atual do projeto.

1. Arquitetura em Camadas (completa e funcional)

```
Controllers (API v1) → Services → Repositories → Models (SQLAlchemy)
                    ↑ Schemas (Pydantic) para validação/serialização
                    ↑ Core / Internal para infraestrutura transversal
```

- ✓ Separação clara de responsabilidades por camada
- ✓ Imports padronizados sob o namespace `src.app.*`

2. Aplicação FastAPI (`src/main.py`) — funcional

- ✓ App configurado com título, descrição e versão (`1.0.0`)
- ✓ **CORS** habilitado
- ✓ **Middleware de logging (AOP)** — `logging_aspect` loga método, rota, status e latência; adiciona header `X-Process-Time-ms`
- ✓ **Métricas Prometheus** expostas em `GET /metrics`
- ✓ **Lifespan**: criação best-effort de tabelas + start/stop do consumer RabbitMQ em background task
- ✓ Endpoints utilitários: `GET /` e `GET /health`
- ✓ **Tratamento global de exceções**: `HTTPException`, `RequestValidationError` (422) e `Exception` genérica (500)
- ✓ Todos os routers registrados sob `/api/v1`

3. Endpoints da API (todos registrados em `/api/v1`)

Módulo	Prefixo	Funcionalidades
Auth	<code>/auth</code>	register, login (OAuth2), refresh, logout, <code>/me</code>
Users	<code>/users</code>	gestão de usuários
Games	<code>/games</code>	cadastro e disponibilidade de jogos
Parties	<code>/parties</code>	criar, listar (todas/abertas/minhas), obter, atualizar, aprovar (admin), cancelar, deletar
Party Members	<code>/parties/{id}/members</code>	solicitar entrada, listar, aprovar/rejeitar, remover, sair
Messages	<code>/parties</code>	troca de mensagens nas parties
Audit	<code>/audit</code>	consulta de logs de auditoria

4. Segurança / JWT (`src/app/core/security.py`) — funcional

- ✓ Hash e verificação de senha (`hash_password` , `verify_password`) com bcrypt
- ✓ Geração de token JWT (`create_access_token`)
- ✓ Decodificação/validação de token (`decode_access_token`)
- ✓ Dependency `get_current_user` (valida JWT, busca usuário no banco, 401/403)
- ✓ **RBAC** via `require_role` (controle de acesso por papel — ex.: ADMIN)
- ✓ Fluxo OAuth2 Password (`/auth/login`)

5. AOP — Aspect-Oriented Programming (`src/app/core/aspects.py`) — funcional

- ✓ `audit_action(entity, action)` — JoinPoint After Returning
- ✓ `log_execution` — JoinPoint Around
- ✓ `require_active_user` — JoinPoint Before
- ✓ `require_role(allowed_roles)` — autorização por papel, JoinPoint Before
- ✓ Middleware de logging atuando como aspecto transversal em `main.py`

6. Mensageria RabbitMQ (`src/app/internal/rabbitmq.py`) — funcional

- ☒ **Producer:** `publish_event` publica eventos de domínio (JSON) com `aio-pika`
 - ☒ **Consumer:** `start_consumer` / `stop_consumer` rodando como background task, processando a fila `ludoteca.events`
 - ☒ Catálogo de eventos em `src/app/internal/events.py` (`LudotecaEvent`):
 - Usuário: `USER_CREATED` , `USER_DEACTIVATED`
 - Party: `PARTY_CREATED` , `PARTY_APPROVED` , `PARTY_REJECTED` , `PARTY_CANCELLED`
 - Membros: `MEMBER_REQUESTED` , `MEMBER_APPROVED` , `MEMBER_REJECTED` , `MEMBER_REMOVED` , `MEMBER_LEFT`
 - Mensagens: `MESSAGE_SENT`
 - Jogos: `GAME_AVAILABILITY_CHANGED`
 - Auditoria: `AUDIT_ACTION`
 - ☒ Eventos integrados nos services (`party_service` , `party_member_service`)
-

7. Design Patterns — implementados

- ☒ **Repository** (`BaseRepository` + repositórios por entidade)
 - ☒ **Factory** (`ServiceFactory` / `get_factory`)
 - ☒ **Dependency Injection** (FastAPI `Depends`)
 - ☒ **Service Layer**
 - ☒ **DTO / Schema** (Pydantic)
 - ☒ **AOP / Decorators**
 - ☒ **Middleware / Interceptor**
 - ☒ **Publish/Subscribe (Event-Driven)**
 - ☒ **Singleton** (Settings via `pydantic-settings`)
-

8. Persistência de Dados — funcional

- ☒ Modelos SQLAlchemy: `user` , `game` , `party` , `party_member` , `message` , `audit_log`
 - ☒ Repositórios com CRUD genérico + consultas específicas (ex.: parties por organizador, contagem de membros aprovados, conflito de horários, próximo organizador elegível)
 - ☒ Schema SQL completo em `docs/database/schema.sql` (inclui ENUM de auditoria com ação `CANCEL`)
 - ☒ Schema aplicado automaticamente no boot do container MySQL
-

9. Infraestrutura / Docker — funcional

- ☒ **Dockerfile** multi-stage (builder + runtime, usuário não-root, healthcheck, porta 8000)
- ☒ **.dockerignore** para imagem enxuta
- ☒ **docker-compose.yml** com 5 serviços orquestrados:

- `api` (FastAPI) — porta 8000
- `mysql` 8.0 — porta 3306, com healthcheck e init do schema
- `rabbitmq` 3.12-management — portas 5672 / 15672, com healthcheck
- `prometheus` — porta 9090
- `grafana` — porta 3000
- ☒ Rede dedicada `ludoteca_net` e volumes persistentes
- ☒ `depends_on` com `condition: service_healthy`
- ☒ **.env.example** documentado

10. Observabilidade — funcional

- ☒ Métricas da API expostas em `/metrics` (prometheus-fastapi-instrumentator)
- ☒ **Prometheus** configurado para scrape de `api:8000` (`infra/prometheus/prometheus.yml`)
- ☒ **Grafana** com datasource Prometheus provisionado (uid fixo `prometheus`)
- ☒ **Dashboard “Ludoteca API”** provisionado (`ludoteca_api.json`) com 3 painéis:
- Requisições por segundo (por handler)
- Latência média das requisições
- Total de requisições por status HTTP

11. Testes — funcionais

- ☒ `pytest.ini` configurado (testpaths, naming, verbose)
- ☒ `conftest.py` : banco **SQLite em memória** (StaticPool), override de `get_db` , **mock do publish_event** (não exige RabbitMQ), helpers de auth
- ☒ `test_auth.py` : register (sucesso, e-mail duplicado 409, e-mail inválido 422), login (sucesso, senha errada 401), `/me` (com/sem token)
- ☒ `test_parties.py` : criação de party, status inicial PENDING, 401 sem token, jogo inexistente 404, `/parties/my` , `/parties/open` , RBAC em `/parties/` (403 para não-admin)
- ☒ Rodam isolados, sem necessidade de infraestrutura externa

12. Documentação — disponível

- ☒ `README.md` — visão geral, setup, execução, URLs, credenciais, patterns
- ☒ `PROJETO_COMPLETO.md` — este inventário
- ☒ Swagger UI (`/docs`) e ReDoc (`/redoc`) gerados automaticamente
- ☒ `docs/` com arquitetura, modelagem, diagramas, requisitos e schema do banco

Como validar rapidamente

```
cd infra && docker-compose up -d --build    # sobe todo o stack
# Acesse http://localhost:8000/docs (Swagger)
# Acesse http://localhost:3000 (Grafana – admin/admin)
# Acesse http://localhost:15672 (RabbitMQ – guest/guest)

cd .. && pytest                                # roda os testes
```

Estado geral:  Projeto completo e funcional ponta a ponta.